

STT / TTS / LLM Cost Tracking — Implementation Notes

How per-call cost is measured, calculated, and logged for the three paid services in the Farm Vaidya voice agent

Project: pipecat · Prepared: 7 August 2026 · Scope: bot_processors/latency.py, Bot.py's pipeline wiring, .env pricing config

1. Files involved

File	Role
bot_processors/latency.py	All the actual logic — state, metric capture, cost formulas, currency formatting, call-end log lines. Everything described in this document lives here unless stated otherwise.
Bot.py	Instantiates the tracker objects, wires them into the pipeline at specific positions, and — critically — enables the pipecat pipeline flags that make usage-metrics data flow at all (see §5).
.env	The only place per-unit prices and the USD→INR rate are configured. No prices are hardcoded in Python.

2. Data flow, end to end

```
Pipecat pipeline                                LatencyLogger (x2 instances)    CallCostTracker

STT service (Soniox)
  emits MetricsFrame (audio_seconds)    ---> catches STTUsageMetricsData
                                           sums into state.stt_usage_seconds

Vertex LLM service (Gemini)
  emits MetricsFrame (prompt/completion tokens) ---> catches LLMUsageMetricsData
                                           sums into state.llm_prompt_tokens /
                                           state.llm_completion_tokens

TTS service (Bakbak)
  emits TTSAudioRawFrame (real PCM audio, every chunk) ---> computes chunk_seconds =
                                           frame.num_frames / frame.sample_rate
                                           sums into state.tts_audio_seconds

                                           at call end:
                                           log_summary() reads
                                           all three totals from
                                           the SAME shared state
                                           object, multiplies by
                                           the configured prices,
                                           logs 3 lines
```

All three trackers share one `_LatencyState` instance (`latency.py`), passed in from `Bot.py` — this is why a single object accumulates STT seconds, LLM tokens, and TTS seconds across the whole call regardless of which pipeline stage is currently writing to it.

3. Shared state — `_LatencyState`

A plain Python class with `__slots__` (not a dataclass — predates this cost-tracking work, kept consistent with the existing latency-measurement fields already in the file). The four fields this document is about:

Field	Type	Populated by
<code>stt_usage_seconds</code>	float	<code>LatencyLogger</code> , from <code>STTUsageMetricsData</code>
<code>llm_prompt_tokens</code>	int	<code>LatencyLogger</code> , from <code>LLMUsageMetricsData</code>
<code>llm_completion_tokens</code>	int	<code>LatencyLogger</code> , from <code>LLMUsageMetricsData</code>
<code>tts_audio_seconds</code>	float	<code>LatencyLogger</code> , computed directly from <code>TTSAudioRawFrame</code> PCM data

4. STT cost

4.1 Capturing usage

Relies on pipecat's own built-in usage-metrics feature (added in `pipecat-ai` 1.7.0). Soniox's STT service internally calls `self.start_stt_usage_metrics(STTUsage(audio_seconds=...))` once per final transcript (plus a trailing flush on stop). If enabled (see §5), this produces a `MetricsFrame(data=[STTUsageMetricsData(value=STTUsage(audio_seconds=X))])` that flows down the pipeline like any other frame.

```
# LatencyLogger.process_frame() – bot_processors/latency.py
elif isinstance(frame, MetricsFrame):
    for d in frame.data:
        if isinstance(d, STTUsageMetricsData):
            self._s.stt_usage_seconds += d.value.audio_seconds
```

4.2 Cost formula

```
# CallCostTracker.log_summary()
stt_cost = (stt_seconds / 60.0) * STT_PRICE_PER_MIN
```

`STT_PRICE_PER_MIN` is read from `.env` — entered directly in ₹ (native), no currency conversion applied. Current value `0.1906`, derived from Soniox's real published rate (\$0.002/min for the `stt-rt-v5` model this project uses) converted at the checked exchange rate.

5. LLM cost

5.1 Capturing usage

Same `MetricsFrame` mechanism as STT. `GoogleLLMService` (the base class `GoogleVertexLLMService` / `_FastVertexLLMService` extends — confirmed by reading pipecat's own source before relying on this) calls `self.start_llm_usage_metrics(LLMTokenUsage(prompt_tokens=..., completion_tokens=..., total_tokens=...))` after every generation. These are Gemini's own real reported token counts, not an estimate.

```
elif isinstance(d, LLMUsageMetricsData):
    self._s.llm_prompt_tokens += d.value.prompt_tokens
    self._s.llm_completion_tokens += d.value.completion_tokens
```

5.2 Cost formula

```
llm_cost = (
    (prompt_tokens / 1_000_000) * LLM_PRICE_PER_1M_INPUT_TOKENS
    + (completion_tokens / 1_000_000) * LLM_PRICE_PER_1M_OUTPUT_TOKENS
)
```

`LLM_PRICE_PER_1M_INPUT_TOKENS` / `_OUTPUT_TOKENS` are the **one exception** to "prices are entered in ₹" — these stay in USD (Google AI Studio's own published rate for `gemini-2.5-flash`: \$0.30 in / \$2.50 out per 1M

tokens), then get converted to ₹ only at display time via `USD_TO_INR_RATE`. Kept in USD because that's genuinely the currency Google's own pricing page quotes; STT/TTS are native ₹ because those vendors' rates were given directly in rupees.

6. TTS cost

Not character-based. An earlier version of this tracker counted LLM output characters and multiplied by a per-1000-character rate — the right model for a provider like Cartesia, but **wrong for Bakbak**, which bills 39 paise per *minute of audio*, not per character. Character count is still logged for visibility, but no longer drives the cost figure.

6.1 Capturing usage — real audio duration, not an estimate

Computed directly from the actual PCM audio bytes in every `TTSAudioRawFrame`, using the same duration formula pipecat itself uses internally (`num_frames = len(audio) / (num_channels × 2)` for 16-bit PCM):

```
# LatencyLogger.process_frame() — runs on EVERY TTS audio chunk,
# not just the first one (unlike the pre-existing first-audio
# latency measurement, which only needs to fire once per turn)
elif isinstance(frame, TTSAudioRawFrame):
    chunk_seconds = frame.num_frames / frame.sample_rate if frame.sample_rate else 0.0
    self._s.tts_audio_seconds += chunk_seconds
```

6.2 Cost formula

```
tts_cost = (tts_seconds / 60.0) * TTS_PRICE_PER_MIN
```

`TTS_PRICE_PER_MIN` = `0.39` (₹, native — the confirmed Bakbak rate). A legacy character-based fallback (`DEFAULT_PRICE_PER_1K`, from `TTS_PRICE_PER_1K_CHARS` / `CARTESIA_PRICE_PER_1K_CHARS`) is still checked if `TTS_PRICE_PER_MIN` is ever unset, for providers that genuinely bill per character.

7. Currency handling — two formatting functions

Function	Used for	Behavior
<code>_format_native_cost(rupees)</code>	STT, TTS	Just formats with a ₹ prefix — no math. The price knobs for these are already entered directly in rupees.
<code>_format_cost(usd)</code>	LLM only	Multiplies by <code>USD_TO_INR_RATE</code> and shows ₹ if that rate is configured (>0); otherwise falls back to showing \$ plain.

Why the split: forcing Bakbak's already-rupee rate through a USD round-trip would be pointless extra math and a source of rounding drift — it's simpler and more accurate to just enter what you actually see on the vendor's bill.

8. The critical enabling fix — `enable_usage_metrics`

Found via log inspection, not assumption: a real call showed the TTS cost line working correctly, but **zero** STT or LLM usage-metric debug lines anywhere in the log — even though `enable_metrics=True` had been set in `Bot.py` the whole time.

Root cause: pipecat's `FrameProcessor` gates `start_stt_usage_metrics` / `start_llm_usage_metrics` behind **two independent flags**, not one:

```
# pipecat/processors/frame_processor.py (installed package, not this project's code)
async def start_llm_usage_metrics(self, tokens: LLMTokensUsage):
    if self.can_generate_metrics() and self.usage_metrics_enabled:
        frame = await self._metrics.start_llm_usage_metrics(tokens)
        if frame:
            await self.push_frame(frame)
```

`can_generate_metrics()` is tied to `PipelineParams.enable_metrics` (already on, which is why TTFB/latency metrics always worked). `usage_metrics_enabled` is tied to a **separate** field, `PipelineParams.enable_usage_metrics`, which defaults to `False` and was never being set. Fixed in `Bot.py`:

[illegible]

9. Pipeline wiring — where these objects live

```
_lat_state    = _LatencyState()                # shared state
latency_early = LatencyLogger(state=_lat_state)  # placed right after
STT
latency_late  = LatencyLogger(state=_lat_state, serializer=...) # placed right after
TTS
cost_tracker  = CallCostTracker(state=_lat_state, ...)          # placed between LLM
and TTS

pipeline = Pipeline([
    ..., latency_early, ...,      # sees TranscriptionFrame, InterimTranscriptionFrame
    ..., cost_tracker,            # sees LLMTextFrame (char count, first-token time)
    ..., latency_late, ...,       # sees TTSAudioRawFrame (both latency AND
duration/cost)
])

# At call end (on_client_disconnected / idle-timeout / escalation paths):
cost_tracker.log_summary()        # reads all three totals, logs 3 lines
```

Both `LatencyLogger` instances and `CallCostTracker` share the exact same `_lat_state` object — this is the only reason a single `log_summary()` call, made once from the TTS-side tracker, can report on STT/LLM/TTS totals that were actually written by three different pipeline stages.

10. Libraries / pipecat internals used

Import	From	Purpose
<code>MetricsFrame</code>	<code>pipecat.frames.frames</code>	Generic carrier frame for any metrics payload
<code>STTUsageMetricsData</code> , <code>LLMUsageMetricsData</code>	<code>pipecat.metrics.metrics</code>	Typed payloads inside a <code>MetricsFrame</code>
<code>TTSAudioRawFrame</code>	<code>pipecat.frames.frames</code>	Real synthesized audio chunks — <code>num_frames</code> / <code>sample_rate</code> read directly off this
<code>FrameProcessor</code> , <code>FrameDirection</code>	<code>pipecat.processors.frame_processor</code>	Base class both trackers extend
<code>PipelineParams</code>	<code>pipecat.pipeline.task</code>	Where <code>enable_metrics</code> / <code>enable_usage_metrics</code> are set
<code>os.getenv</code>	stdlib	All price/rate configuration, no hardcoded defaults for real vendor prices
<code>time.monotonic()</code>	stdlib	Elapsed-call-time for the "₹/min" rate figures
<code>loguru.logger</code>	third-party (already a project dependency)	All the log output described in §11

No external HTTP calls happen inside the cost-tracking code itself — all pricing is static config read from `.env` ; only the underlying pipecat services (Soniox, Vertex, Bakbak) make their own network calls, and this code only observes the metrics/audio frames they already produce.

11. Current .env configuration

Variable	Value	Currency	Source
<code>USD_TO_INR_RATE</code>	95.28	—	xe.com mid-market rate, checked 2026-08-07 (drifts daily — re-check periodically)
<code>STT_PRICE_PER_MIN</code>	0.1906	₹ (native)	Soniox real-time streaming, <code>stt-rt-v5</code> : \$0.12/hr → \$0.002/min → ×95.28
<code>TTS_PRICE_PER_MIN</code>	0.39	₹ (native)	Confirmed Bakbak rate (39 paise/min)
<code>LLM_PRICE_PER_1M_INPUT_TOKENS</code>	0.30	\$ (converted for display)	Google AI Studio, <code>gemini-2.5-flash</code>
<code>LLM_PRICE_PER_1M_OUTPUT_TOKENS</code>	2.50	\$ (converted for display)	Google AI Studio, <code>gemini-2.5-flash</code>
<code>TTS_PRICE_PER_1K_CHARS</code>	(blank)	—	Legacy fallback, unused while <code>TTS_PRICE_PER_MIN</code> is set

12. Worked calculation examples

Full step-by-step arithmetic for each of the three cost types, using the same numbers that appear in §13's log output — so every figure below can be checked directly against a real or verified-test log line.

12.1 STT — Soniox

```
Given:
  stt_usage_seconds      = 300.0 s    (5 minutes of audio submitted)
  STT_PRICE_PER_MIN      = ₹0.1906   (native rupees, no conversion)

Formula:
  stt_cost = (stt_usage_seconds / 60) × STT_PRICE_PER_MIN

Calculation:
  stt_cost = (300.0 / 60) × 0.1906
           = 5.0 × 0.1906
           = ₹0.9530                ← matches §13's logged figure exactly
```

12.2 TTS — Bakbak

```
Given:
  tts_audio_seconds      = 66.0 s    (real PCM duration, from an actual call)
  TTS_PRICE_PER_MIN      = ₹0.39     (native rupees – 39 paise/min)

Formula:
  tts_cost = (tts_audio_seconds / 60) × TTS_PRICE_PER_MIN

Calculation:
  tts_cost = (66.0 / 60) × 0.39
           = 1.1 × 0.39
           = ₹0.4290                ← logged as ₹0.4291; the 1-paise
                                   difference is float rounding on the
                                   unrounded audio_seconds value (the
                                   log line only displays it to 1
                                   decimal place, "66.0s")
```

The accompanying `~₹0.1851/min` figure in that same log line is a **different** rate from the ₹0.39/min billing price — it's `total_cost ÷ elapsed CALL minutes` (wall-clock call duration, 2.3 min), not cost per minute of audio. It answers "how fast is this call accumulating cost" rather than "what's the per-minute billing rate" — the two will only match if TTS audio filled 100% of the call with no silence, pauses, or STT/thinking time in between.

12.3 LLM — Gemini (gemini-2.5-flash)

```
Given:
llm_prompt_tokens      = 1500      (real count from Gemini, via LLMUsageMetricsData)
llm_completion_tokens  = 400
LLM_PRICE_PER_1M_INPUT_TOKENS = $0.30  (USD — Google's own published rate)
LLM_PRICE_PER_1M_OUTPUT_TOKENS = $2.50
USD_TO_INR_RATE        = 95.28

Formula:
input_cost_usd = (prompt_tokens / 1,000,000) × LLM_PRICE_PER_1M_INPUT_TOKENS
output_cost_usd = (completion_tokens / 1,000,000) × LLM_PRICE_PER_1M_OUTPUT_TOKENS
llm_cost_usd   = input_cost_usd + output_cost_usd
llm_cost_inr   = llm_cost_usd × USD_TO_INR_RATE

Calculation:
input_cost_usd = (1500 / 1,000,000) × 0.30 = 0.0015 × 0.30 = $0.00045
output_cost_usd = (400 / 1,000,000) × 2.50 = 0.0004 × 2.50 = $0.00100
llm_cost_usd   = 0.00045 + 0.00100 = $0.00145

llm_cost_inr   = 0.00145 × 95.28
                = ₹0.138156
                ≈ ₹0.1382      ← matches §13's logged figure exactly
```

This is the only one of the three where a currency conversion actually happens — STT and TTS above never leave rupees at any point in the calculation.

13. Sample real output

From an actual call (not a synthetic test) — `logs/call_919949070894_21316ba4.log`:

```
📞 CALL END | BAKBAK: 66.0s audio (726 chars) ₹0.4291 total  call 2.3 min  ~₹0.1851/min
```

The STT/LLM lines were absent from this particular call because it predates the §8 fix — expected format, once `enable_usage_metrics` takes effect on a fresh call:

```
🎙️ CALL END | SONIOX: 300.0s audio submitted ₹0.9530 total
🗨️ CALL END | gemini-2.5-flash: 1500 in / 400 out tokens ₹0.1382 total
```

(These two lines are from a controlled test reproducing realistic call volumes, verified to match hand-calculated expected values exactly — not fabricated log output. Same numbers as §12's worked examples.)